

Fundamentos del Aprendizaje Automático

Clase 1 · Herramientas, conceptos y primeros modelos

Agenda de hoy

01

Motivación

¿Por qué ML? Un problema concreto de inicio

02

Ecosistema Python

NumPy · Pandas ·
Scikit-learn ·
Matplotlib

03

El ciclo ML

Representar →
Entrenar → Evaluar
→ Refinar

04

Conceptos clave

Datos, modelos,
parámetros,
hiperparámetros

05

Overfitting & Underfitting

Generalización y
cómo diagnosticarla

Motivación

¿Por qué necesitamos machine learning?

Problema de arranque: predicción de precios de vivienda

Tenemos datos de 1000 inmuebles en Medellín:

m² construidos	Estrato	Barrio	Antigüedad (años)	Precio (millones COP)
85	4	Laureles	10	320
120	5	El Poblado	3	580
55	2	Belén	20	180
200	6	Envigado	1	?

¿Cómo estimamos el precio del último inmueble?

Respuesta clásica: regla de negocio manual (si $m^2 > 100$ y estrato $\geq 5 \rightarrow$ precio alto...) ❌ frágil

Respuesta ML: aprender la relación directamente de los datos ✓

Tipos de aprendizaje automático

Supervisado

- Aprendemos de datos ETIQUETADOS
- Cada ejemplo tiene una respuesta correcta
- Regresión → predice un número continuo
- Clasificación → predice una categoría
- Ejemplo: precio de vivienda, diagnóstico médico

No supervisado

- Aprendemos de datos SIN etiquetar
- El algoritmo encuentra estructura por sí solo
- Clustering → agrupa observaciones similares
- Reducción de dimensionalidad (PCA, t-SNE)
- Ejemplo: segmentación de clientes

Ecosistema Python para ML

Las herramientas que usaremos en el curso

El stack de Python para Machine Learning



NumPy

Matrices y álgebra lineal eficiente. Es la base de scikit-learn.
`import numpy as np`



Pandas

DataFrames para cargar, limpiar y explorar datos tabulares.
`import pandas as pd`



Scikit-learn

Algoritmos de ML listos para usar: entrenar, evaluar, refinar.
`from sklearn...`



Matplotlib / Seaborn

Visualización de datos y resultados del modelo.
`import matplotlib.pyplot as plt`

Importaciones estándar — las usaremos en cada sesión

```
# Manipulación de datos
import numpy as np
import pandas as pd

# Visualización
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# Reproducibilidad — siempre fijar la semilla
np.random.seed(42)
```



Copiar estas líneas al inicio de cada notebook del curso

El ciclo de ML

Representar → Entrenar → Evaluar → Refinar

El ciclo de vida de un modelo ML

1. DATOS

Recolectar
Limpiar
Explorar (EDA)

2. REPRESENTAR

Features
Normalizar
Codificar

3. ENTRENAR

Elegir modelo
Ajustar parámetros
(train set)

4. EVALUAR

Métricas
Diagnóstico
(test set)

5. REFINAR: ajustar hiperparámetros, nuevas features, más datos → volver al paso 3

Estructura de los datos — la matriz X y el vector y

X (features / variables)

m ²	Estrato	Antigüedad
85	4	10
120	5	3
55	2	20
...	...	...

y (etiqueta / target)

Precio (M COP)
320
580
180
...

m filas (muestras)

n columnas (features)

Regresión → y es continua (precio)

Clasificación → y es categórica (aprobado/rechazado)

División entrenamiento / prueba — la regla de oro

ENTRENAMIENTO (80%)

PRUEBA (20%)

- ✓ El modelo SOLO ve datos de entrenamiento durante el ajuste
- ✓ Los datos de prueba simulan datos nuevos del mundo real
- ✗ **Nunca uses datos de prueba para tomar decisiones de diseño**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Conceptos clave del aprendizaje

Parámetros, hiperparámetros y optimización

Parámetros vs. Hiperparámetros

Parámetros

- Se aprenden AUTOMÁTICAMENTE durante el entrenamiento
- Son internos al modelo
- Ejemplos en regresión lineal: coeficientes $w_0, w_1, \dots, w_n$
- Ejemplos en red neuronal: pesos de cada neurona
- No los tocamos directamente

Hiperparámetros

- Los fijamos ANTES de entrenar
- Controlan la forma/complexidad del modelo
- Árbol de decisión: profundidad máxima (max_depth)
- KNN: número de vecinos k
- Se seleccionan por validación cruzada

Optimización — cómo aprende el modelo

- El aprendizaje = encontrar los parámetros que MINIMIZAN una función de pérdida (loss function)
- Función de pérdida para regresión: $MSE = (1/m) \sum (\hat{y}_i - y_i)^2$
 - Para clasificación: log-loss, cross-entropy
- Algoritmos de optimización más comunes:
 - Gradiente Descendente: moverse en la dirección que reduce el error
 - Solución analítica (solo regresión lineal): $w = (X^T X)^{-1} X^T y$
 - Búsqueda en grilla / aleatoria para hiperparámetros
- Como estadísticos, ya conocen estimación por MV — la optimización ML es la misma idea

Generalización

Overfitting · Underfitting · Diagnóstico

Los tres escenarios de ajuste

Underfitting

Modelo demasiado simple

Error entreno: Alto
Error prueba: Alto

Solución:
Aumentar complejidad
Más features

Buen ajuste

Modelo generaliza bien

Error entreno: Bajo
Error prueba: Bajo

Solución:
¡Este es el objetivo!

Overfitting

Modelo demasiado complejo

Error entreno: Muy bajo
Error prueba: Alto

Solución:
Regularización
Más datos
Menos features

¿Cómo diagnosticar overfitting/underfitting?

- Comparar el error en entrenamiento vs. el error en prueba:
 - Error train bajo + Error test ALTO → Overfitting
 - Error train ALTO + Error test alto → Underfitting
 - Error train bajo + Error test bajo → Bien calibrado ✓
- Herramienta clave: curvas de aprendizaje (learning curves)
 - Graficar error vs. tamaño del conjunto de entrenamiento
 - Si las curvas convergen: buen ajuste; si divergen: overfitting
- Validación cruzada (cross-validation) da una estimación más robusta → lo vemos en Clase 2

Resumen de la Clase

- ML aprende relaciones directamente de los datos, sin reglas manuales
- El ciclo Representar → Entrenar → Evaluar → Refinar es el marco de trabajo
- Siempre separar datos de entrenamiento y prueba ANTES de ajustar
- Parámetros se aprenden; hiperparámetros se diseñan
- Overfitting = memorizar; underfitting = ignorar; el objetivo es generalizar

Notebook de práctica disponible — ejecutar durante la sesión y completar los ejercicios 